

LAB 11

COMPUTER STORE MANAGEMENT & REVIEW SYSTEM

I. MỤC TIÊU

Sinh viên cần xây dựng hệ thống backend:

- Xác thực người dùng (JWT)
- Quản lý sản phẩm máy tính
- Hệ thống đánh giá sản phẩm
- Thực hiện truy vấn nâng cao (Populate + Aggregation)

II. YÊU CẦU CHUNG

Project: ComputerStore_StudentCode

Công nghệ:

- Node.js
- Express.js
- MongoDB + Mongoose
- JWT
- bcrypt
- dotenv

III. DATABASE STRUCTURE

1. Users

```
{
  "username": "alice",
  "password": "$2b$10$hashed",
  "role": "user",
  "createdAt": "2026-03-10T08:30:00Z"
}
```

2. Products

```
{
  "name": "MacBook Pro M2",
  "brand": "Apple",
  "category": "Laptop",
  "price": 45000000,
  "stock": 5,
  "specifications": {
    "cpu": "M2",
    "ram": "16GB",
    "storage": "512GB SSD"
  },
  "imageUrl": "/images/macbook.jpg",
  "averageRating": 4.8,
  "createdAt": "2026-03-01T08:00:00Z"
}
```

3. Reviews

```
{
  "productId": "ObjectId",
  "userId": "ObjectId",
  "rating": 5,
  "comment": "Excellent performance!",
  "createdAt": "2026-03-05T08:00:00Z"
}
```

IV. CẤU HÌNH HỆ THỐNG

Tạo file .env:

PORT=1234

MONGO_URI=mongodb://127.0.0.1:27017/ComputerStoreDB_2026

JWT_SECRET=computer_store_secret

V. NỘI DUNG

TASK 1 — Project Setup (10 điểm)

- Tạo database: ComputerStoreDB_2026
- Kết nối MongoDB
- Tạo structure project (MVC)
- Chạy server thành công

TASK 2 — Register API

Endpoint:

POST /users/register

Yêu cầu:

- Kiểm tra trùng username
- Hash password (bcrypt)
- Role mặc định: user

TASK 3 — Login API (10 điểm)

POST /users/login

Yêu cầu:

- Kiểm tra username/password
- Trả JWT token (expire 24h)

```
{  
  "token": "JWT_TOKEN"  
}
```

TASK 4 — Get Products (10 điểm)

GET /products

Yêu cầu:

- Public API
- Trả danh sách sản phẩm
- Sort theo newest

TASK 5 — Product Details (Populate)

GET /products/:id

Yêu cầu:

- Xác thực JWT
- Populate:
 - Reviews
 - Username

TASK 6 — Create Product (Admin)

POST /products

Yêu cầu:

- Chỉ admin
- Validate dữ liệu

TASK 7 — Update Product (Admin)

PUT /products/:id

TASK 8 — Delete Product (Admin)

DELETE /products/:id

TASK 9 — Add Review (10 điểm)

POST /reviews/:productId

Yêu cầu:

- User đăng nhập
- 1 user chỉ review 1 lần
- Update averageRating

TASK 10 — My Reviews (Populate)

GET /reviews/myreviews

Yêu cầu:

Populate:

- name
- brand
- category

TASK 11 — Top Rated Products (Aggregation)

GET /products/top-rated

- Sort rating giảm dần
- Limit 10

TASK 12 — Most Reviewed Products (Aggregation)

GET /products/most-reviewed

TASK 13 — Product Statistics

GET /products/:id/statistics

Kết quả:

```
{
  "averageRating": 4.5,
  "totalReviews": 20,
  "ratingDistribution": {
    "5": 10,
    "4": 5,
    "3": 3,
    "2": 1,
    "1": 1
  }
}
```

VI. SECURITY REQUIREMENTS

- JWT Authentication:

Authorization: Bearer <token>

- Middleware:

- authMiddleware
- roleMiddleware (admin/user)

DATABASE

🌿 Tổng quan hệ thống

Hệ thống gồm 3 collections:

- **Users** → quản lý tài khoản
- **Products** → quản lý sản phẩm máy tính
- **Reviews** → đánh giá sản phẩm

👉 Quan hệ:

Users (1) — (N) Reviews (N) — (1) Products

1. USERS COLLECTION

Field	Type	Required	Description
_id	ObjectId	✓	ID tự động
username	String	✓	Tên đăng nhập (unique)
password	String	✓	Mã hóa bcrypt
role	String	✓	user / admin
createdAt	Date	✓	Ngày tạo

👉 Kiến thức:

- Unique index cho username
- Không lưu password dạng plain text
- Dùng bcrypt để hash

2. PRODUCTS COLLECTION

Field	Type	Required	Description
_id	ObjectId	✓	Product ID
name	String	✓	Tên sản phẩm
brand	String	✓	Hãng
category	String	✓	Laptop / PC / Accessory
price	Number	✓	Giá
stock	Number	✓	Số lượng
specifications	Object	✓	CPU, RAM, Storage
imageUrl	String	✗	Ảnh
averageRating	Number	✓	Tính từ reviews
createdAt	Date	✓	Ngày tạo

👉 Kiến thức:

- Schema lồng (nested object)
- Không nhập tay averageRating (tính bằng code)
- Dùng sort, filter

3. REVIEWS COLLECTION

Field	Type	Required	Description
_id	ObjectId	✓	Review ID
productId	ObjectId	✓	Ref → Products
userId	ObjectId	✓	Ref → Users
rating	Number	✓	1 → 5
comment	String	✗	Nhận xét
createdAt	Date	✓	Thời gian

👉 Kiến thức:

- Quan hệ reference (ObjectId)
- Populate:

- productId → product info
- userId → username
- 1 user chỉ review 1 product

IV. RELATIONSHIP (HIỂU RÕ)

Relation	Type	Meaning
User → Reviews	1 → N	1 user nhiều review
Product → Reviews	1 → N	1 product nhiều review
Review → User	N → 1	mỗi review thuộc 1 user
Review → Product	N → 1	mỗi review thuộc 1 product

V. SAMPLE DATA (IMPORT FILES)

File 1 — users.json

```
[
  {
    "_id": { "$oid": "66f100000000000000000001" },
    "username": "alice",
    "password": "$2b$10$hashedpassword1",
    "role": "user",
    "createdAt": { "$date": "2026-03-01T08:00:00Z" }
  },
  {
    "_id": { "$oid": "66f100000000000000000002" },
    "username": "bob",
    "password": "$2b$10$hashedpassword2",
    "role": "user",
    "createdAt": { "$date": "2026-03-02T09:00:00Z" }
  },
  {
    "_id": { "$oid": "66f100000000000000000003" },
    "username": "charlie",
    "password": "$2b$10$hashedpassword3",
    "role": "user",
    "createdAt": { "$date": "2026-03-03T10:00:00Z" }
  },
  {
    "_id": { "$oid": "66f100000000000000000004" },
    "username": "admin",
    "password": "$2b$10$hashedadmin",
    "role": "admin",
    "createdAt": { "$date": "2026-03-01T07:00:00Z" }
  }
]
```

File 2 — products.json

```
[
  {
    "_id": { "$oid": "66f200000000000000000001" },
    "name": "Dell XPS 13",
    "brand": "Dell",
    "category": "Laptop",
    "price": 35000000,
    "stock": 10,
    "specifications": {
      "cpu": "Intel i7",
      "ram": "16GB",
      "storage": "512GB SSD"
    },
    "imageUrl": "/images/dellxps13.jpg",
    "averageRating": 4.8,
    "createdAt": { "$date": "2026-03-01T08:00:00Z" }
  },
  {
    "_id": { "$oid": "66f200000000000000000002" },
    "name": "MacBook Pro M2",
    "brand": "Apple",
    "category": "Laptop",
    "price": 45000000,
    "stock": 5,
    "specifications": {
      "cpu": "M2",
      "ram": "16GB",
      "storage": "512GB SSD"
    },
    "imageUrl": "/images/macbook.jpg",
  }
]
```

```

    "averageRating": 4.7,
    "createdAt": { "$date": "2026-03-02T08:00:00Z" }
  },
  {
    "_id": { "$oid": "66f200000000000000000003" },
    "name": "Asus Gaming ROG",
    "brand": "Asus",
    "category": "Laptop",
    "price": 40000000,
    "stock": 7,
    "specifications": {
      "cpu": "Intel i9",
      "ram": "32GB",
      "storage": "1TB SSD"
    },
    "imageUrl": "/images/asusrog.jpg",
    "averageRating": 4.6,
    "createdAt": { "$date": "2026-03-03T08:00:00Z" }
  },
  {
    "_id": { "$oid": "66f200000000000000000004" },
    "name": "Logitech MX Master 3",
    "brand": "Logitech",
    "category": "Accessory",
    "price": 2500000,
    "stock": 20,
    "specifications": {
      "type": "Mouse",
      "connectivity": "Wireless"
    },
    "imageUrl": "/images/mouse.jpg",
    "averageRating": 4.5,
    "createdAt": { "$date": "2026-03-04T08:00:00Z" }
  }
]

```

File 3 — reviews.json

```

[
  {
    "_id": { "$oid": "66f300000000000000000001" },
    "productId": { "$oid": "66f200000000000000000001" },
    "userId": { "$oid": "66f100000000000000000001" },
    "rating": 5,
    "comment": "Very powerful and compact laptop!",
    "createdAt": { "$date": "2026-03-05T08:00:00Z" }
  },
  {
    "_id": { "$oid": "66f300000000000000000002" },
    "productId": { "$oid": "66f200000000000000000001" },
    "userId": { "$oid": "66f100000000000000000002" },
    "rating": 4,
    "comment": "Great design but a bit expensive.",
    "createdAt": { "$date": "2026-03-05T09:00:00Z" }
  },
  {
    "_id": { "$oid": "66f300000000000000000003" },
    "productId": { "$oid": "66f200000000000000000002" },
    "userId": { "$oid": "66f100000000000000000001" },
    "rating": 5,
    "comment": "Excellent performance and battery life.",
    "createdAt": { "$date": "2026-03-06T10:00:00Z" }
  },
  {
    "_id": { "$oid": "66f300000000000000000004" },
    "productId": { "$oid": "66f200000000000000000003" },
    "userId": { "$oid": "66f100000000000000000003" },

```

```

    "rating": 5,
    "comment": "Perfect for gaming!",
    "createdAt": { "$date": "2026-03-07T11:00:00Z" }
  },
  {
    "_id": { "$oid": "66f300000000000000000005" },
    "productId": { "$oid": "66f200000000000000000004" },
    "userId": { "$oid": "66f100000000000000000002" },
    "rating": 3,
    "comment": "Good but a bit overpriced.",
    "createdAt": { "$date": "2026-03-08T12:00:00Z" }
  },
  {
    "_id": { "$oid": "66f300000000000000000006" },
    "productId": { "$oid": "66f200000000000000000003" },
    "userId": { "$oid": "66f100000000000000000001" },
    "rating": 4,
    "comment": "Strong performance but heavy.",
    "createdAt": { "$date": "2026-03-08T13:00:00Z" }
  }
]

```

VI. HƯỚNG DẪN IMPORT

✓ Cách 1 — MongoDB Compass

1. Tạo DB: ComputerStoreDB_2026
2. Tạo collections:
 - users
 - products
 - reviews
3. Import JSON từng file

✓ Cách 2 — Mongo Shell

```
mongoimport --db ComputerStoreDB_2026 --collection users --file users.json --
jsonArray
```

```
mongoimport --db ComputerStoreDB_2026 --collection products --file
products.json --jsonArray
```

```
mongoimport --db ComputerStoreDB_2026 --collection reviews --file reviews.json
--jsonArray
```


TESTCASE

Task 1: Project Setup

Project folder: ComputerStore_StudentCode
Database name: ComputerStoreDB_2026
Collections: Users, Products, Reviews
Server URL: http://localhost:1234

Task 2: API – POST /users/register

Endpoint

POST http://localhost:1234/users/register

Body → raw → JSON

```
{
  "username": "usera",
  "password": "123456"
}
```

Expected Result

- User created
- Password hashed

```
{
  "message": "User registered successfully"
}
```

Task 3: API – POST /users/login

Endpoint

POST http://localhost:1234/users/login

Body

```
{
  "username": "usera",
  "password": "123456"
}
```

Expected Result

```
{
  "token": "JWT_TOKEN"
}
```

 Save token

Task 4: API – POST /users/logout

Endpoint

POST http://localhost:1234/users/logout

Headers

Authorization: Bearer <token>

Expected Result

```
{
  "message": "Logged out successfully"
}
```

Task 5: API – GET /products

Endpoint

GET http://localhost:1234/products

Expected Result

```
[
  {
    "_id": "66f200000000000000000001",
    "name": "Dell XPS 13",
    "brand": "Dell",
  }
]
```

```
    "category": "Laptop",
    "price": 35000000,
    "averageRating": 4.8
  }
]
```

Task 6: API – GET /products/:id (Authenticated + Populate)

Endpoint

GET http://localhost:1234/products/66f200000000000000000001

Headers

Authorization: Bearer <token>

Expected Result

```
{
  "_id": "66f200000000000000000001",
  "name": "Dell XPS 13",
  "brand": "Dell",
  "reviews": [
    {
      "rating": 5,
      "comment": "Very powerful!",
      "user": {
        "username": "alice"
      }
    }
  ]
}
```

Task 7: API – POST /products (Admin only)

Endpoint

POST http://localhost:1234/products

Headers

Authorization: Bearer <admin_token>

Body

```
{
  "name": "HP Pavilion",
  "brand": "HP",
  "category": "Laptop",
  "price": 20000000,
  "stock": 8,
  "specifications": {
    "cpu": "Intel i5",
    "ram": "8GB",
    "storage": "256GB SSD"
  }
}
```

Expected Result

```
{
  "message": "Product created successfully"
}
```

Task 8: API – PUT /products/:id

Endpoint

PUT http://localhost:1234/products/66f200000000000000000001

Headers

Authorization: Bearer <admin_token>

Body

```
{
  "price": 34000000,
  "stock": 12
}
```

Expected Result

```
{
  "message": "Product updated successfully"
}
```

Task 9: API – DELETE /products/:id

Endpoint

DELETE http://localhost:1234/products/66f20000000000000000000000000004

Headers

Authorization: Bearer <admin_token>

Expected Result

```
{
  "message": "Product deleted successfully"
}
```

Task 10: API – POST /reviews/:productId

Endpoint

POST http://localhost:1234/reviews/66f20000000000000000000000000002

Headers

Authorization: Bearer <token>

Body

```
{
  "rating": 5,
  "comment": "Excellent laptop!"
}
```

Expected Result

```
{
  "message": "Review added successfully"
}
```

Task 11: API – PUT /reviews/:id

Endpoint

PUT http://localhost:1234/reviews/66f30000000000000000000000000001

Headers

Authorization: Bearer <token>

Body

```
{
  "rating": 4,
  "comment": "Updated review after using longer."
}
```

Expected Result

```
{
  "message": "Review updated successfully"
}
```

Task 12: API – DELETE /reviews/:id

Endpoint

DELETE http://localhost:1234/reviews/66f30000000000000000000000000001

Headers

Authorization: Bearer <token>

Expected Result

```
{
  "message": "Review deleted successfully"
}
```

Task 13: API – GET /reviews/myreviews

Endpoint

GET http://localhost:1234/reviews/myreviews

Headers

Authorization: Bearer <token>

Expected Result

```
{
  "user": "usera",
  "reviews": [
    {
      "product": {
        "name": "MacBook Pro M2",
        "brand": "Apple",
        "category": "Laptop"
      },
      "rating": 5,
      "comment": "Excellent!",
      "createdAt": "2026-03-20T10:00:00Z"
    }
  ]
}
```

Task 14: API – GET /products/top-rated

Endpoint

GET <http://localhost:1234/products/top-rated>

Expected Result

```
[
  {
    "name": "Dell XPS 13",
    "averageRating": 4.8
  }
]
```

Task 15: API – GET /products/most-reviewed

Endpoint

GET <http://localhost:1234/products/most-reviewed>

Expected Result

```
[
  {
    "_id": "66f200000000000000000001",
    "reviewCount": 2
  }
]
```

Task 16: API – GET /products/:id/statistics

Endpoint

GET <http://localhost:1234/products/66f200000000000000000001/statistics>

Expected Result

```
{
  "averageRating": 4.8,
  "totalReviews": 2,
  "ratingDistribution": {
    "5": 1,
    "4": 1,
    "3": 0,
    "2": 0,
    "1": 0
  }
}
```

BONUS TESTCASE (OPTIONAL)

◆ Search Products

GET <http://localhost:1234/products?brand=Apple>

◆ Filter Price

GET <http://localhost:1234/products?minPrice=20000000&maxPrice=40000000>

